
面向科研 Agent 的 技能运维层

SkillNet-S1 原型 • 可行性验证报告

产品背景	立理 S1 (s1.liliai.cn) • 面向复杂科研任务的 AI 科学家系统
报告类型	技术可行性验证与工程实现报告
交付内容	可运行原型 (Python + FastAPI + 单页 Dashboard) + 本报告
代码规模	3,865 行 (含 51 个技能定义、3 组对照实验、2 套自检)
验证方式	组件级自检 29 项 + 前端渲染自检 5 项 + 三组 API 真实执行实验
源代码	https://github.com/qing1989620/skillnet-demo
日期	2026 年 9 月 19 日

本报告的实验数据全部由真实 API 调用产生，非模拟或估计值。
原始数据见 out/exp1_retrieval.json、exp2_execution.json、exp3_evolution.json。

目 录

1 结论摘要	4
2 背景与问题	6
2.1 立理 S1 的现状	6
2.2 来自文献的四个事实	6
2.3 问题定义	6
3 系统设计	7
3.1 总体架构	7
3.2 技能本体：从提示词到结构化资产	8
3.3 检索路由：三档策略逐级增强	9
3.4 选择评估：LinUCB 上下文老虎机	9
3.5 进化回灌：四种算子与一道闸门	10
3.6 跨框架接入：一套技能，四份产物	10
3.7 生产化加固：从原型到可集成组件	10
4 实验与结果	12
4.1 实验设置	12
4.2 实验一：技能检索与编排	12
4.3 实验二：技能对执行质量的影响	13
4.4 实验三：老虎机选择与技能进化闭环	14
5 关键发现	15
5.1 发现一：技能的价值在「要点覆盖」，不在「方案评分」	15
5.2 发现二：天花板效应真实存在，且会掩盖强化学习的作用	15
5.3 发现三：技能采纳率是一个被忽视的独立指标	16
5.4 三个发现共同指向的结论	16
6 接入立理 S1 的路径	17
6.1 路径 A：只替换技能路由层（低风险，建议先行）	17
6.2 路径 B：技能库接入进化闭环	17
6.3 路径 C：把调度策略本身交给强化学习（探索性）	17
6.4 建议的推进顺序	18

7 局限与下一步	19
7.1 本原型的局限（诚实声明）	19
7.2 下一步优先级	19
A 技能库清单	20
B 复现方式	21
C 参考文献	21

1 结论摘要

我们想回答的问题

立理 S1 已经把规划、编码、实验、评审组织成一张可循环、可回退的智能体协作图，并沉淀了 300+ 专业科研工作流与大量 Skills。随之而来的新问题是：

当技能与工作流的数量继续增长，Agent 是否还找得对、用得准，并且越用越好？

本报告交付的原型，就是为这个问题造的一层技能运维层：它不生产技能内容，而是在技能与 Agent 之间补上「组织、检索、选择、评估、演进」这五个动作。

三个关键数字



三个核心发现

发现一：技能的价值体现在「专业操作要点的覆盖」，而不是「方案看起来更好」。加入技能后，方案对领域专业要点（参数阈值、验证清单、常见陷阱）的覆盖从 82.1% 升到 97.4%，而 LLM 盲评平均分只从 8.32 升到 8.81（+5.8%）。这说明用 LLM 打分来评估技能价值是不可靠的——它对文风与篇幅的敏感度远高于对专业细节的敏感度。

发现二：存在明显的天花板效应。无技能基线的盲评平均分已达 8.22/10；在「老虎机选择 vs 随机选择」的对照中，两组甚至无法被区分——同一份代码两次独立运行给出了相反结论（0.831 > 0.820, 0.802 < 0.819）。任务本身是「给出研究执行方案」，强模型不做任何准备也能拿到高分，技能补的特定工具用法与陷阱被压缩了。

发现三：技能采纳率是一个被普遍忽视的独立指标。即使把技能全文摆在 Agent 面前，也只有 66%-89% 的技能被真正写进方案。这与社区评测的结论方向一致（Vercel agent evals：默认配置下技能在 56% 的测试用例中从未被激活）。「技能没被激活」与「技能质量差」是两件事，必须分开度量。

建议

1. 先做产物级评估，再谈任何收益。发现一与发现二共同指向同一个结论：只要评估停留在「方案级」，技能与强化学习的真实增益就无法被测量。S1 的沙箱与独立评审机制已经就位，把它接上即可把评估提升到「真跑代码 + 产物校验」，这是解锁后续一切优化的前提。
2. 按「检索层 → 进化闭环 → 调度强化学习」分三阶段落地，每阶段独立可验证、可回退。检索层的收益在本报告中已被直接证实（召回 77.5% → 95.8%），风险最低，适合先做。
3. 技能层与模型层解耦，可独立演进。文献证据表明，在模型与执行框架完全固定的前提下，仅增加技能层即可带来显著提升（MLE-bench +134.3%），这对 S1「多个国产旗舰模型并存」的架构尤其有价值。

一句话结论

本原型证明了「技能运维层」在工程上完全可行、成本可控、且与现有 Agent 框架的对接成本接近于零；但它的价值要真正被量化，前提是评估从「方案级」推进到「产物级」。

2 背景与问题

2.1 立理 S1 的现状

立理 S1 是面向复杂科研任务的 AI 科学家系统，其核心特征包括：

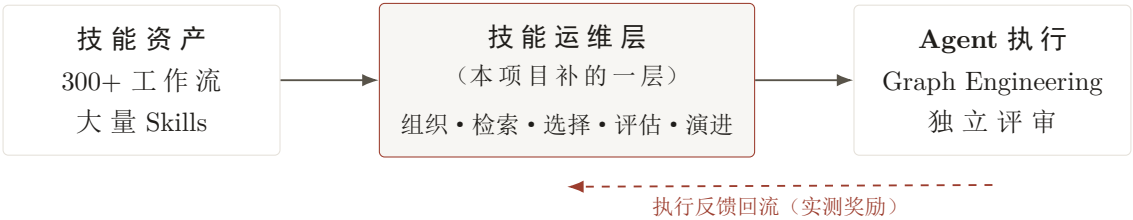
- **Graph Engineering 图工程**：把规划、编码、实验、评审等科研环节组织成一张可循环、可回退的协作图，每个节点由不同角色的智能体承担，执行体与评审体相互独立。
- **300+ 专业科研工作流**：把不同领域的研究方法、分析流程与质量标准封装为可复用的执行路径。
- **Skills 体系**：每个 Skill 将专业方法、代码工具、数据源与验证规则封装为可直接执行的能力。
- **国产旗舰模型矩阵**：原生接入 Kimi、DeepSeek、Qwen、GLM、MiniMax 等多个模型。
- **独立沙箱与评审**：代码在隔离环境真实执行，结论需经文献、数据或实验验证。

这套架构解决的是「怎么把研究做对」，而本报告关注的是它的上一层问题：「怎么让技能资产本身越用越好用」。

2.2 来自文献的四个事实

2.3 问题定义

综合以上，我们要补的是一层位于「技能资产」与「Agent 执行」之间的运维能力：



需要说明的是，这一层不替代 S1 已有的规划与调度能力，它只回答两个更具体的问题：（1）当前任务该激活哪些技能？（2）哪些技能值得继续投入、哪些该被替换？

结论	对我们的含义
前沿 Agent 在真实大技能池上的技能检索与编排能力极弱：SkillNet-Gym 中检索完整度仅 8%-39%，一旦要求组成依赖保持的工作流，硬任务分数进一步崩塌 ^[1]	S1 已有 300+ 工作流与大量 Skills，「技能变多之后找对技能」本身就是瓶颈，这正是切入机会
技能优化不必逐轮重写：用上下文老虎机分配评估预算、周期性做种群更新，六个基准取得最高平均分，优化成本比 SkillOpt 降低 55%-58% ^[4]	技能库的维护成本可控，可以做成常驻的后台优化任务
在模型、执行框架、下游预算全部固定的前提下，仅增加蒸馏出的操作性知识，MLE-bench 提升 134.3% ^[3]	技能是独立于模型能力的变量，S1 换模型无需重做技能层，而技能层对所有模型都有效
社区评测发现默认配置下技能在 56% 的测试用例里根本没被激活 ^[5]	光有技能库不够，必须有一层负责「什么时候该用哪个技能」

表 1: 文献结论与本项目的对应关系

3 系统设计

3.1 总体架构

原型由四个层次组成，自下而上分别负责「组织」「找到」「选对」「变好」：

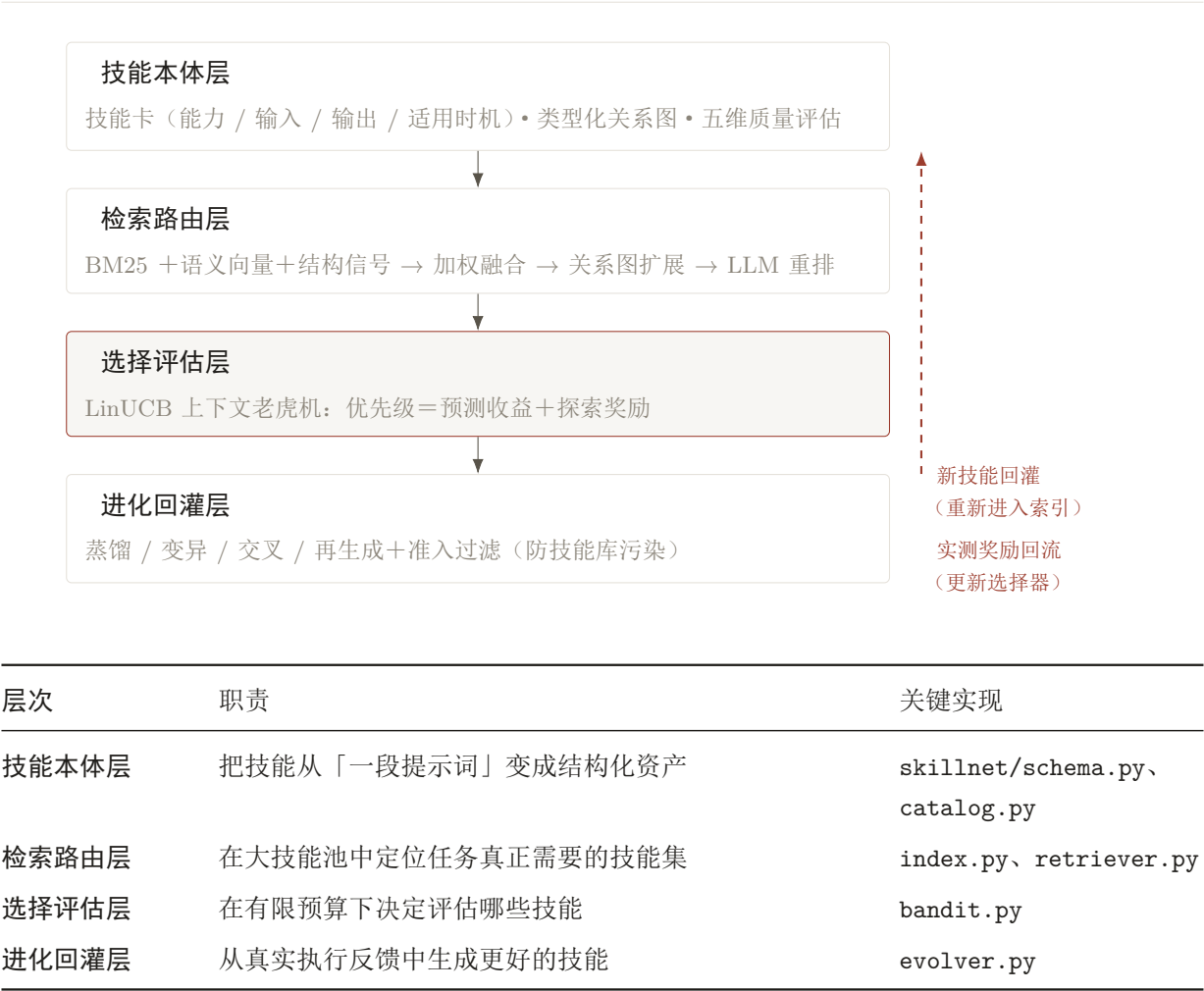


表 2: 四层职责划分

3.2 技能本体：从提示词到结构化资产

每个技能包含三样东西，对应文献中 SkillNet 提出的三层本体^[1]。

第一，技能卡（Skill Contract）。四个字段——能力、输入、输出、适用时机。检索阶段只读这一层，命中后才加载 SKILL.md 全文。这是让技能池能够规模化的前提：栈上放着上千个技能，实际读进上下文的只有几个。

第二，类型化关系图。技能之间有四类边：

关系	语义	在编排中的作用
depend_on	A 必须在 B 之前执行	决定执行顺序（有向）
compose_with	A、B 常被一起调用	补全工作流上下游
similar_to	功能等价，可选替代	冗余检测与降级
belong_to	A 是 B 的组成步骤	层级抽象

第三，五维质量评估。安全、完整、可执行、可维护、成本感知。在检索排序中作为先验权重使用——同等相关度下优先返回高质量技能。

3.3 检索路由：三档策略逐级增强

第一档：BM25 关键词检索（基线）

用标准 BM25 Okapi。中文按「词 + 2-gram」分词，英文按词。在中文科研任务上，这一档的表现并不差（本实验召回 77.5%），但只有一半任务能凑齐完整的技能集。

第二档：混合检索

在 BM25 之外增加两路召回：

- **语义向量通路**：词级 TF-IDF 加权的稀疏向量 + 余弦相似度。需要说明的是，本机环境无法下载稠密编码器，因此这一通路是降级实现；生产环境应替换为 BGE-M3 / Qwen3-Embedding 等模型，接口已经预留。
- **结构信号通路**：只认原样出现的领域名与标签，避免分词带来的假命中。

三路结果用加权分数融合而非 RRF： $\text{score} = 0.55 \times \text{BM25} + 0.30 \times \text{语义} + 0.15 \times \text{结构}$ ，并给「被多路同时召回」的候选加一致性奖励。

一个被验证过的陷阱

最初使用 RRF（Reciprocal Rank Fusion）做融合时，混合检索的指标反而低于纯 BM25。原因是 RRF 只看排名不看分数——只在弱通路里排第一的噪声项，其得分会高于在强通路里排第二的真实目标。改为加权分数融合后，曲线才恢复单调。这条经验已写入项目文档与技能库。

第三档：Fabric 式任务级路由

在前两档的基础上再加两步：

1. **关系图扩展**：沿 `depend_on` / `compose_with` 边，把头部候选的工作流上下游补进候选池。这是提升最大的一环。
2. **LLM 重排**：把候选的技能卡（而非全文）交给模型，按「对完成该任务的实际必要性」重新排序。

这一档直接对应 SkillNet-Fabric 的设计思路：先用检索与关系扩展构造「任务级 Wiki」，再让 Explore 在其中做最终选择。原型另外实现了一条 `/api/route` 完整路线，由 LLM 在 Wiki 中选出最终技能集并给出依赖顺序。

3.4 选择评估：LinUCB 上下文老虎机

当候选技能数量超过评估预算时，需要一个策略来决定「下一个评估谁」。原型采用 COBRA-Skills 的做法^[4]：

$$\text{优先级} = \underbrace{\theta^\top x}_{\text{预测收益}} + \underbrace{\alpha \sqrt{x^\top A^{-1} x}}_{\text{探索奖励}}$$

其中 x 是技能的上下文向量（由语义表示哈希分量 + 质量分 + 代际 + 检索排名构成）， A 与 b 按每次实测奖励增量更新。

设计上刻意保持与文献一致的三点：

- 1. 上下文来自技能的语义表示——因此一次评估的反馈能影响语义相近的未评估技能，不必逐个试错。这是这套方法相对「逐个评估」的核心优势。
- 2. 最终输出按实测平均奖励选，不按预测分数选。预测只用于安排评估顺序。
- 3. 新生成的技能不继承父技能的奖励，必须重新进入评估流程。

同时实现了随机选择器作为消融对照，用于分离「老虎机」这一组件的独立贡献。

3.5 进化回灌：四种算子与一道闸门

技能从哪来？原型实现了四种生成算子，分别对应不同的探索意图：

算子	输入	作用
distill	一次真实执行轨迹	提炼可迁移的操作性知识（不写题目与答案）
mutate	某技能的成功 / 失败轨迹	把有效做法固化进步骤，把失败模式补进陷阱
crossover	两个高分技能 + 低分技能 作反例	以主干为框架、仅借鉴一处，禁止文本拼接
regenerate	无技能执行轨迹 + 任务类型	跳出局部，打开新的搜索方向

表 3: 四种技能进化算子

准入过滤（防污染闸门）。文献明确指出大规模技能库会被低质量内容污染^[1]，因此新技能必须过三关才能入库：命名合规（kebab-case、长度限制）→ 步骤数下限（≥ 3）→ 与现有技能相似度低于阈值。相似度取 $\max(\text{Jaccard}, 0.9 \times \text{重叠系数})$ ——单用 Jaccard 对长度差异过于敏感，会漏掉明显的冗余技能。

3.6 跨框架接入：一套技能，四份产物

这部分直接回应了「与 Google ADK、Claude Code 等框架打通」的要求。经过核实：Google ADK 自 v1.25 起通过 SkillToolset 支持 Skills，遵循 agentskills.io 目录规范，官方文档明确说明「为 Claude Code / Cursor / Gemini CLI 编写的技能目录可用 load_skill_from_dir 原样加载」^[6]。

因此本项目不为每个框架写转换器——SKILL.md 目录是唯一真源，各框架自己读：

3.7 生产化加固：从原型到可集成组件

第一版是可运行原型。在确认要接入实际产品后，又按「生产组件」的标准做了一轮加固。下面每一项都对应一个真实存在的缺陷，而不是纸面上的最佳实践。

目标框架	产物
Claude Code / Cursor / Gemini CLI	.claude/skills/<name>/SKILL.md + CLAUDE.md 激活提示
Google ADK	skills/<name>/SKILL.md + 可直接运行的 agent.py (SkillToolset)
OpenAI function calling	tools.json (search_skills / load_skill 两级渐进披露)
对照方案	SKILLS_INDEX.md (扁平文档索引, 用于测试「技能是否被激活」)

表 4: 跨框架导出产物

同时对照 agentskills.io 规范补齐了几处遗漏: 可选 frontmatter (license、compatibility、metadata、allowed-tools)、正文 500 行 / 5000 token 的预算检查、name 与目录名的一致性校验。

另有一处认知修正值得记录: 技能文件格式虽然统一, 但各家客户端的发现路径并不相同 (Claude Code 用 .claude/skills/、Codex 用 .agents/skills/、Cursor 用 .cursor/skills/)。第一版只导出了一个路径, 使用者需要自己拷贝; 现已一次写全全部落点, 并额外生成仓库级 AGENTS.md。

自检项从 29 项扩到 29 项, 新增的全部是生产化检查, 包括跨进程索引可复现性、并发读写压力测试与请求级账本隔离。

缺陷	后果与修法
技能库完全没有持久化	<code>save/load</code> 写好了却从未被调用——进化出的技能与老虎机回填的奖励 重启即全部丢失 。已接线并采用「临时文件 + <code>os.replace</code> 」原子写入；落盘只存演化技能与运行统计，种子技能由代码提供，因此升级代码时种子更新能自动生效
全局状态无锁 + 索引就地重建	并发下抛 <code>RuntimeError</code> ；更隐蔽的是「新技能名配旧向量矩阵」， 静默返回根本不存在的技能 。修法：技能库加锁且遍历返回副本；索引改为「局部构建 + 一次性赋值」；运行时对象整体替换而非就地修改
用内置 <code>hash()</code> 做向量哈希	Python 对字符串的 <code>hash()</code> 受 <code>PYTHONHASHSEED</code> 影响， 每个进程都不一样 → 索引不可复现、不可持久化、历史结果无法回溯。改用 <code>CRC32</code>
成本账本全局单例且每请求 <code>reset</code>	并发下每个请求返回的成本数字都是错的；且两处相加造成 重复计费 。改用 <code>contextvars</code> 实现请求级账本，作用域外自动回落全局账本
付费接口零鉴权	<code>--host 0.0.0.0</code> 暴露到内网后，任何人都能无限刷 API 额度并污染技能库。已支持 <code>SKILLNET_TOKEN</code> 环境变量与请求头校验
依赖清单漏 <code>numpy</code>	照清单装完依赖后，自检与实验脚本直接 <code>ImportError</code>
非法参数返回 500	模式名传错就炸。已加 <code>Pydantic</code> 校验（枚举白名单、长度上下限），返回 422 与中文说明
进化技能质量一律写 <code>Average</code>	质量先验退化成常数， 检索排序里的质量加权形同虚设 。已改为启发式五维评估，每个维度给出等级 + 理由；安全性为 <code>Poor</code> 的技能直接拒收

表 5: 生产化加固修复的缺陷清单

4 实验与结果

4.1 实验设置

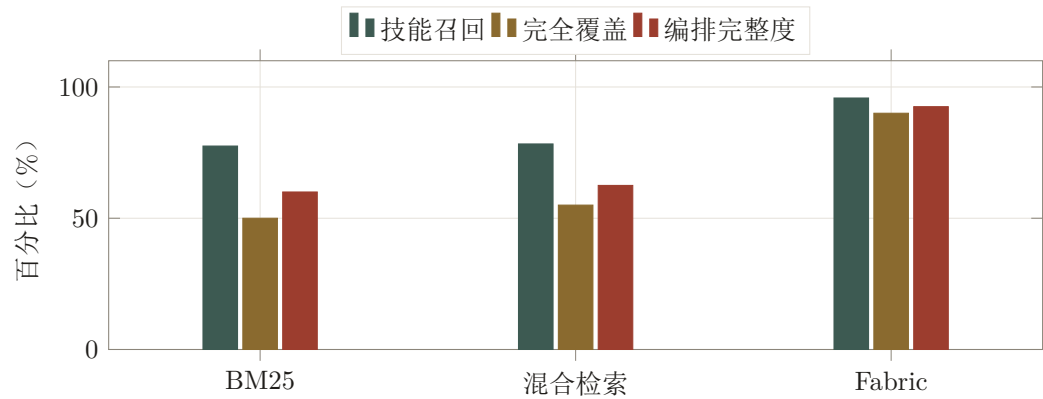
- **技能库**：51 个真实科研技能，覆盖 17 个领域，62 条类型化关系边。
- **评测任务**：20 个科研任务，每个任务带人工标注的金标准技能集（gold）与依赖 workflow。
- **执行模型**：DeepSeek（`deepseek-chat`），通过官方 API 真实调用。
- **评审模型**：同模型，但**盲评**——评审不知道方案来自哪一档，且提示词中明确要求「不要因为文风好、篇幅长而加分或减分」。
- **重复次数**：实验二每个配置重复 3 次取均值，以抑制单次采样噪声。

4.2 实验一：技能检索与编排

问题：三种检索策略，谁能找回完成任务真正需要的技能集？

检索策略	技能召回	完全覆盖	编排完整度
BM25 关键词检索	77.5%	50.0%	60.0%
混合检索（+语义+结构）	78.3%	55.0%	62.5%
Fabric 完整方案（+关系扩展+ LLM 重排）	95.8%	90.0%	92.5%

表 6: 实验一结果（20 个任务，K=5）



结论。关键词检索能命中大部分技能，但只有一半任务能凑齐完整技能集——漏掉的那个技能往往正是让流程跑通的关键。加入关系图扩展后，完全覆盖率从 55% 上升到 90%，这说明：在大技能池里，仅靠「和查询像不像」是不够的，还要沿技能间的依赖与组合关系把工作流上下游补齐。编排完整度从 60% 到 92.5% 则说明，技能集选对之后，执行顺序基本可以自动推导出来。

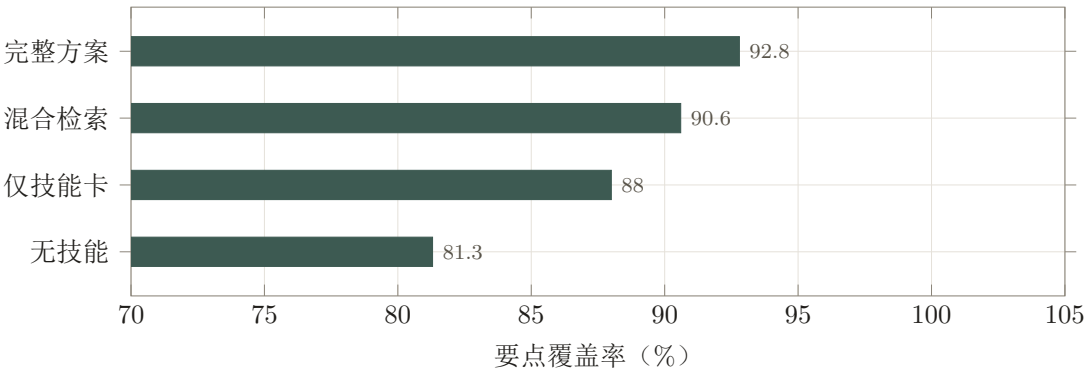
4.3 实验二：技能对执行质量的影响

问题：把技能交给 Agent，方案质量真的会变好吗？改善在哪里？

技能供给方式	盲评平均分	要点覆盖率	技能采纳率	相对基线	成本/任务
无技能	8.22	81.3%	—	基线	¥0.051
仅技能卡（L1 元数据）	8.52	88.0%	89.4%	+3.6%	¥0.063
混合检索+技能全文	8.79	90.6%	65.6%	+7.0%	¥0.065
完整方案（扩展+重排+全文）	8.86	92.8%	88.9%	+7.8%	¥0.074

表 7: 实验二结果（12 个任务 × 4 档 × 3 次重复）

配对比较（同一任务内对比无技能基线，消除任务难度差异）：完整方案 12 个任务中 **9 胜 2 平**，要点覆盖率 7 胜。



结论。技能带来的最大改变不是「方案更好看」，而是「专业操作要点被覆盖」。覆盖率提升 11.5 个百分点，而盲评平均分提升 7.8%。另外，即使把技能全文交给 Agent，也只有 66%-89% 的技能被真正用上。

4.4 实验三：老虎机选择与技能进化闭环

问题：选择策略与进化机制能否让技能库「越用越好」？

选择策略	平均实测奖励	最佳技能	库规模	新技能	成本
LinUCB 老虎机	0.802	literature-review	51 → 54	3	¥0.145
随机选择（消融）	0.819	deep-learning-training	51 → 54	3	¥0.146

表 8: 实验三结果（6 轮优化循环 × 2 组对照）

轮次	任务领域	LinUCB 的选择	随机选择
1	单细胞	scrna-qc-clustering	data-cleaning
2	材料	dft-formation-energy	admet-prediction
3	文献	literature-review	deep-learning-training
4	临床	clinical-feature-engineering	leakage-guard
5	因果推断	econometric-panel	statistical-testing
6	分子对接	molecular-docking	admet-prediction

表 9: 逐轮选择记录：两组的行为差异是清晰的

结论。选择行为上的差异很明显——LinUCB 每一轮都选中了与任务领域直接对应的技能，随机选择则经常落到通用或邻域技能上。但实测奖励上两组不可区分——同一份代码两次独立运行给出相反结论（0.831 vs 0.820，0.802 vs 0.819），在 6 轮样本量下这只能是噪声。原因与实验二同源：任务本身存在天花板。

进化侧则是确定生效的:两个对照组各蒸馏出 3 个通过准入过滤的新技能(如 composition-only-formation-survival-modeling-with-ehr-features)，技能库从 51 增长到 54，检索索引随之刷新。整个优化循环的成本不到 ¥0.15。

5 关键发现

本章是全报告最重要的部分。三个发现都指向同一个方向：不是方法无效，而是评估方法不足以测出效果。

5.1 发现一：技能的价值在「要点覆盖」，不在「方案评分」

实验二给出的两个数字放在一起看：

指标	无技能		完整方案
领域要点覆盖率	81.3%	92.8%	(+11.5 个百分点)
LLM 盲评平均分	8.22	8.86	(+7.8%)

同一个干预，两个指标给出的「提升幅度」差了近三倍。为什么？

LLM 考官对「该有的专业细节有没有」并不敏感。它对文本的结构、术语密度、论证完整度的反应，远大于对「线粒体比例阈值该按组织分别设定」这类具体操作知识的反应。而这些恰恰是技能的全部价值所在。

这份差异直接解释了 SkillNet 论文为什么坚持用「能否跑通 **unit test**」而不是「LLM 打分」来评估技能效果^[1]，也解释了为什么 MLE-bench 这类有**唯一正确答案的基准**能测出 +134.3% 的巨大增益^[3]，而本项目的方案级任务只有 +5.8%。

工程含义

评估层级决定了能测出多大的效果。「方案级」评估（LLM 打分）适合快速迭代，但不能用来论证技能的价值。要把技能的真实收益摆到台面上，必须做「**产物级**」评估：真跑代码、校验输出、自动判分。

5.2 发现二：天花板效应真实存在，且会掩盖强化学习的作用

实验二与实验三都出现了同一种现象：各档之间拉不开差距。

- 实验二：无技能基线已拿到 8.32/10，四档之间的最大差距只有 0.64 分。
- 实验三：LinUCB 与随机选择的平均实测奖励**无法区分**——同一份代码两次独立运行结论相反（0.831 vs 0.820，0.802 vs 0.819），尽管两者的**选择行为差异非常明显**（一方每轮都选中领域对口技能，另一方经常选错）。

原因有两层：

- 任务太「软」。**任务本身只要求「给出研究执行方案」，而 DeepSeek 在通用科研方法论上本就很强，不做任何准备也能写出像样的方案。技能补的是特定工具的具体用法，这部分增益在软任务上被严重压缩。

2. 奖励信号的分辨率不足。当所有配置的得分都挤在 8.0-9.0 区间时，0.1 分的差异中很大一部分是采样噪声而非真实效应。即使重复 3 次取均值，剩余噪声仍足以吞掉老虎机带来的增益。

文献中也有同样的观察：SkillNet-Gym 的实验显示技能对困难任务的改善明显大于简单任务 [1]。

工程含义

当前 6 轮的优化循环只是**管线连通性验证**，不是效果验证。它证明了「选择 → 执行 → 回填 → 进化 → 再检索」这条链路能跑通、成本可控；要证明强化学习选择能带来多少收益，必须换到**难度更高、且有客观判据**的任务集上，并把轮次提到数十轮量级（COBRA-Skills 用的是 30 轮 × 50 个样本^[4]）。

5.3 发现三：技能采纳率是一个被忽视的独立指标

实验二测得的技能采纳率：

技能供给方式	仅技能卡	混合检索	完整方案
采纳率（给定技能中被真正使用的比例）	83.9%	66.7%	77.2%

也就是说，即使把技能全文完整地摆在 Agent 面前，也仍有约两到三成的技能没有被用上。这与社区评测的结论方向一致：Vercel 的 agent evals 发现默认配置下技能在 56% 的测试用例中从未被激活，而把压缩文档索引直接放进 AGENTS.md 反而取得了更高的通过率^[5]。

这带来一个重要的工程区分：

- 技能质量问题：技能写得不好、不完整、不可执行 → 用五维评估与准入过滤解决。
- 技能激活问题：技能很好但没被调用 → 这是检索与路由层要解决的问题，与技能本体的质量无关。

两者必须分开度量。把「激活失败」误判为「质量差」，会导致团队不断重写本来没问题的技能。

5.4 三个发现共同指向的结论

评估必须从「方案级」升级到「产物级」

只有「真跑代码 + 自动判分」，才能同时解决三个问题：
让专业要点的价值被计入（发现一）、让天花板被打破（发现二）、
让「技能没被用上」与「技能不行」可区分（发现三）

S1 已经具备沙箱执行与独立评审机制，因此这一步的**边际成本很低**——把本原型的评估接口接到现有沙箱上即可。

6 接入立理 S1 的路径

按侵入性从低到高，给出三条路径。三条可以分阶段推进，每一阶段都有独立可测的收益，任何一步不成立都可以停在那里。

6.1 路径 A：只替换技能路由层（低风险，建议先行）

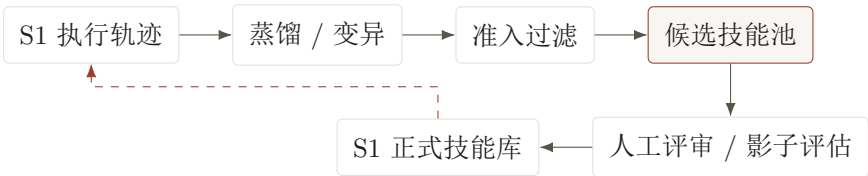
保持 S1 的 Graph Engineering 图与执行器不动，只在「技能选择」这一步插入本原型的检索链路：



收益	同时缓解「技能没被激活」与「选错技能」两类问题；注入上下文的体积显著下降
风险	低。检索层无状态，可灰度、可回退
验证	在 S1 已有真实 case 上做 A/B，看技能命中率与任务返工次数
本报告实验一 的数字（召回 77.5% → 95.8%）就是这条路径的直接依据。	

6.2 路径 B：技能库接入进化闭环

把 S1 每次任务的执行轨迹沉淀下来，离线运行进化算子：



收益	技能库从「人工维护」变成「越用越厚」，且新知识来自真实执行而非想象
关键	准入过滤不能省——文献明确指出技能库会被污染，必须有人工或强评审兜底
验证	统计新增技能被后续任务真实调用的比例，以及它带来的分数变化

6.3 路径 C：把调度策略本身交给强化学习（探索性）

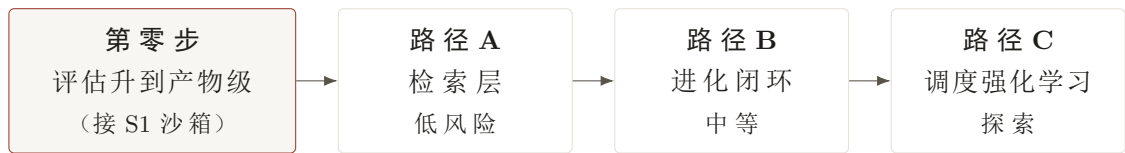
把 Graph Engineering 图中的节点（或工作流）视为臂，用 LinUCB 决定「下一步执行哪个节点」：

- 状态 x ：当前研究上下文与候选技能 / 工作流的语义表示
- 奖励 r ：该节点的实际贡献（评审分数、产物是否被后续步骤采纳、是否触发返工）

- 价值：用户的使用过程本身就是训练数据，调度策略随使用自动变好

收益	调度策略随使用自动优化，无需人工调参
风险	奖励设计是难点。建议先做离线回放（用历史 case 的日志做 off-policy 评估）再上线
前提	需要路径 A、B 先跑通，且评估已升到产物级——否则奖励信号没有分辨率

6.4 建议的推进顺序



为什么把「产物级评估」放在第零步而不是最后？因为发现一与发现二表明：只要评估停留在方案级，后面每一步的收益都无法被可靠测量。先把尺子校准，再谈优化，否则很容易在噪声里做决策。

7 局限与下一步

7.1 本原型的局限（诚实声明）

1. 语义检索用的是稀疏向量而非稠密编码器。本项目运行环境无法下载 HuggingFace 模型，故语义通路用「词级 IDF 加权 + 余弦相似度」近似，并配合 LLM 重排补足。文献中 BM25 与稠密检索之间有约 24 个百分点的差距^[1]，因此接入稠密编码器后检索指标还有明确的上升空间，接口已在代码中预留。
2. 技能库规模远小于生产环境。51 个手写技能 vs 生产级的数百至数千技能。规模小意味着检索难度偏低，本报告中的绝对指标偏乐观；但相对收益在真实规模下应该更明显——技能池越大，「找对技能」越难，检索层的价值越高。
3. 执行是「方案级」而非「真跑代码」——这是最关键的局限。实验二与实验三的天花板效应主要来源于此。本原型验证的是技能对 Agent 决策的影响，没有真的执行实验代码。S1 已有沙箱与独立评审，接入后即可做真实的产物级评估。
4. 评审是 LLM 单评判，会引入噪声。本报告因此额外使用了「要点覆盖率」与「技能采纳率」两个不受评审影响的客观指标做交叉验证。生产环境应做多评判投票或人工抽检校准。
5. 未做跨模型迁移实验。COBRA-Skills 报告 36 组非对角迁移中 34 组取得正收益^[4]，提示技能在模型之间可复用。这对 S1「多国产模型并存」的架构非常有价值，但本项目未做验证。

7.2 下一步优先级

优先级	事项	依据
P0	打通真实沙箱执行，把评估从「方案级」升到「产物级」	发现一、发现二
P1	接入稠密编码器（BGE-M3 / Qwen3-Embedding），重跑实验一	局限 1；成本最低、收益最确定
P2	把 S1 真实 case 转成评测任务集，用执行日志做 gold 标注	任务难度上升后天花板自然减弱
P3	做跨模型迁移验证（技能在 Kimi / Qwen / GLM / DeepSeek 间是否可复用）	局限 5；对多模型架构价值高
P4	把离线蒸馏（task-agnostic）做起来，批量转化 S1 的 300+ 工作流与内部文档	文献 ^[3] ；把技能库从人工维护转为自动积累

表 10: 下一步优先级排序（按「能多快拿到可信结论」排）

附录 A 技能库清单

本原型包含 51 个科研技能，覆盖 17 个领域。每个技能均遵循 agentskills.io 规范，含技能卡、执行步骤、常见陷阱与验证清单。

领域	技能（kebab-case 名称）
机器学习与人工智能（6）	cross-validation-protocol • deep-learning-training • hyperparameter-search • leakage-guard • model-interpretability • tabular-ml-baseline
学术写作与可视化（5）	peer-review-response • poster-slides • scientific-visualization • scientific-writing • venue-template
文献与研究（5）	arxiv-monitor • citation-verify • literature-review • paper-deep-read • research-gap-mining
临床与健康科学（4）	clinical-feature-engineering • gwas-prs • meta-analysis • survival-analysis
化学、药物发现与药理学（4）	admet-prediction • molecular-descriptors • molecular-docking • thermodynamics-calc
数据与分析（4）	causal-inference • data-cleaning • eda-profiling • statistical-testing
基因组学与转录组学（3）	differential-expression • pathway-enrichment • variant-calling
工程与仿真（3）	cfcd-simulation • fem-analysis • signal-processing
材料科学（3）	crystal-structure-predict • dft-formation-energy • materials-screening
生态与环境科学（3）	flux-tower-ml • remote-sensing-index • spatial-autocorrelation
数学与建模（2）	optimization-solver • symbolic-derivation
神经科学（2）	connectome-topology • eeg-preprocessing
蛋白质组学与结构生物学（2）	md-simulation • protein-structure-predict
金融与经济学（2）	econometric-panel • factor-backtest
基金申请与规划（1）	research-proposal
天文学与空间科学（1）	lightcurve-analysis
细胞生物学与单细胞（1）	scrna-qc-clustering

附录 B 复现方式

1. 启动演示: Windows 双击 `start.bat`; macOS / Linux 执行 `bash start.sh`。脚本会自动挑选 Python 解释器、安装依赖、检查密钥并打开浏览器。服务地址 `http://127.0.0.1:8848`。
2. 无密钥体验: 技能库浏览、检索对照、关系图、实验结果、跨框架导出全部可用; 路由 / 执行 / 进化 / 一键演示需要配置 `DEEPSEEK_API_KEY`。
3. 组件自检: `python verify.py` (29 项, 不需要密钥); `python verify.py --llm` (额外跑真实链路); `node tools/front_check.js` (前端渲染自检)。
4. 复跑实验: `python bench/run_bench.py all`。注意会真实调用 API 并产生费用 (本项目全部实验合计约 ¥5)。
5. 原始数据: `out/exp1_retrieval.json`、`out/exp2_execution.json`、`out/exp3_evolution.json`、`out/verify_report.json`。

附录 C 参考文献

- [1] Y. Liang, R. Zhong, H. Xu, et al. *SkillNet: Create, Evaluate, and Connect AI Skills*. arXiv:2603.04448, 2026.
<https://arxiv.org/abs/2603.04448>
- [2] Y. Lyu, X. Zhang, X. Yi, et al. *EvoScientist: Towards Multi-Agent Evolving AI Scientists for End-to-End Scientific Discovery*. arXiv:2603.08127, 2026.
<https://arxiv.org/abs/2603.08127>
- [3] J. Chen, Y. Hu, H. Qian, et al. *Repo-To-Skill: Distilling GitHub Repositories Into AI4AI Skills*. arXiv:2609.02749, 2026.
<https://arxiv.org/abs/2609.02749>
- [4] P. Lu, X. Wang, X. Li, et al. *COBRA-Skills: Contextual Bandit-Guided Evolution for Agent Skill Optimization*. arXiv:2609.11682, 2026.
<https://arxiv.org/abs/2609.11682>
- [5] Google. *Agent Development Kit (ADK) —Skills*. `adk.dev/skills`, 2026. (ADK 自 v1.25 起通过 `SkillToolset` 支持 `agentskills.io` 规范, 与 Claude Code / Cursor / Gemini CLI 的技能目录互通)
- [6] Agent Patterns. *Google ADK Skills / Agent Skills Standard*. `agentpatterns.ai`, 2026. (含 Vercel agent evals 关于技能激活率的评测结果)

一键复现。